

# Guião 06 – SDN com Mininet e Open vSwitch

Infraestruturas de Computação na Nuvem, Aula 06

---

## 1 Objetivos

---

- Observar a rede virtual real do servidor Proxmox (bridges) e do cluster Kubernetes (overlay CNI).
- Criar uma topologia de rede virtual com Mininet, usando switches Open vSwitch (OVS).
- Observar o comportamento da rede sem controller e com um controller OpenFlow ligado.
- Inspeccionar a tabela de fluxos instalada num switch OVS, incluindo prioridades, contadores e timeouts.
- Implementar uma política de rede simples (bloqueio de tráfego entre dois hosts) através do controller.

## 2 Pré-requisitos e Material

---

- A VM `icn-lab` no Proxmox (Guião 01), onde será instalado o Mininet. Recomenda-se pelo menos 2 GB de RAM.
- Acesso à interface web do Proxmox, para a observação inicial da rede do servidor.
- Acesso ao cluster Kubernetes da Aula 05, se ainda estiver montado (opcional, para a Parte 1).
- Acesso root/sudo na VM: o Mininet cria interfaces de rede virtuais e exige privilégios.

**Nota:** o Mininet corre *dentro* da sua VM, criando aí uma rede virtual emulada. Não interfere com a rede do servidor Proxmox nem com a das restantes VMs da turma.

## 3 Introdução

---

Esta aula tem duas partes complementares. Na primeira, observamos redes virtuais *reais* já em funcionamento: as bridges que o Proxmox usa para ligar as VMs da turma, e a rede overlay que o plugin CNI criou no cluster Kubernetes da Aula 05. Na segunda, construímos do zero uma rede definida por software com Mininet e um controller OpenFlow, para ver o mecanismo por dentro.

O Mininet emula hosts, switches e ligações num único Linux, usando os mesmos namespaces de rede estudados na Aula 04. Os switches são instâncias de Open vSwitch – o mesmo software usado em datacenters reais. Vamos primeiro observar que, sem controller, o switch não sabe o que fazer aos pacotes; depois ligamos um controller Ryu que instala as regras na tabela de fluxos; e por fim modificamo-lo para aplicar uma política de rede.

## 4 Parte 1 – Observar redes virtuais reais

---

1. **Observar as bridges do Proxmox.** Na interface web, selecione o *node* e abra *System* → *Network*. Identifique a bridge (tipicamente *vmbr0*) à qual as VMs estão ligadas e registre a que interface física corresponde.

Se tiver acesso à shell do node, observe a mesma informação por CLI:

```
$ ip link show type bridge
$ brctl show 2>/dev/null || bridge link
```

Esta bridge é um switch virtual em software: desempenha, para as VMs, o papel que um switch físico desempenha para servidores físicos.

2. **Observar a rede overlay do Kubernetes** (se o cluster da Aula 05 ainda estiver disponível). A partir do control plane:

```
$ kubectl get pods -o wide -A | head -20
$ ip addr show
```

Procure interfaces criadas pelo plugin CNI (ex. *flannel.1*, *cni0*). A interface *flannel.1* é um **VTEP**: o ponto onde o tráfego entre Pods de nós diferentes é encapsulado em VXLAN.

Compare a sub-rede dos Pods com a sub-rede das VMs:

```
$ kubectl get nodes -o jsonpath='{.items[*].spec.podCIDR}'
```

Registe as duas gamas de endereços. A dos Pods é a rede *overlay*; a das VMs é o *underlay*.

## 5 Parte 2 – Construir uma rede SDN com Mininet

---

3. **Instalar o Mininet e o Ryu** na VM *icn-lab*:

```
$ sudo apt update
$ sudo apt install -y mininet openvswitch-switch python3-pip
$ sudo systemctl enable --now openvswitch-switch
```

O controller Ryu tem dependências sensíveis à versão do Python; instale-o num ambiente virtual para não interferir com o sistema:

```
$ sudo apt install -y python3-venv
$ python3 -m venv ~/ryu-env
$ ~/ryu-env/bin/pip install ryu
```

*Nota:* caso a instalação do Ryu falhe por incompatibilidade de versões (é um projeto pouco mantido), a docente indicará a alternativa a usar – tipicamente uma versão fixada do pacote, ou o controller de referência do próprio Mininet para os passos que não exigem código próprio.

4. Confirmar que o Mininet e o Open vSwitch estão operacionais:

```
$ sudo mn --version
$ sudo ovs-vsctl --version
```

5. Criar uma topologia simples com três hosts ligados a um único switch, sem indicar controller (o Mininet arranca um controller de referência por omissão):

```
$ sudo mn --topo single,3 --mac --switch ovsk
```

Dentro da consola do Mininet, testar a conectividade:

```
mininet> pingall
```

Deve funcionar, pois o controller de referência do Mininet instala automaticamente um comportamento de *learning switch*.

6. Sair do Mininet (`exit`) e repetir a topologia, desta vez sem qualquer controller, para observar o efeito de um switch órfão:

```
$ sudo mn --topo single,3 --mac --switch ovsk --controller none
mininet> pingall
```

Sem controller, o switch não tem forma de aprender para onde encaminhar os pacotes: os pings falham. Isto ilustra a dependência do plano de dados em relação ao plano de controlo.

7. Sair do Mininet. Num terminal separado, criar um controller simples baseado em Ryu, ficheiro `learning_switch.py`:

```
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import MAIN_DISPATCHER, CONFIG_DISPATCHER, set_ev_cls
from ryu.ofproto import ofproto_v1_3
from ryu.lib.packet import packet, ethernet

class LearningSwitch(app_manager.RyuApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(LearningSwitch, self).__init__(*args, **kwargs)
        self.mac_to_port = {}

    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def switch_features_handler(self, ev):
        datapath = ev.msg.datapath
        parser = datapath.ofproto_parser
        ofproto = datapath.ofproto
        match = parser.OFPMatch()
        actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                          ofproto.OFPCML_NO_BUFFER)]
        self.add_flow(datapath, 0, match, actions)

    def add_flow(self, datapath, priority, match, actions):
        parser = datapath.ofproto_parser
        ofproto = datapath.ofproto
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS, actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=priority,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def packet_in_handler(self, ev):
        msg = ev.msg
        datapath = msg.datapath
```

```

ofproto = datapath.ofproto
parser = datapath.ofproto_parser
in_port = msg.match['in_port']

pkt = packet.Packet(msg.data)
eth = pkt.get_protocols(ethernet.ethernet)[0]
dst = eth.dst
src = eth.src
dpid = datapath.id
self.mac_to_port.setdefault(dpid, {})
self.mac_to_port[dpid][src] = in_port

out_port = self.mac_to_port[dpid].get(dst, ofproto.OFPP_FLOOD)
actions = [parser.OFPACTIONOutput(out_port)]

if out_port != ofproto.OFPP_FLOOD:
    match = parser.OFPMATCH(in_port=in_port, eth_dst=dst)
    self.add_flow(datapath, 1, match, actions)

data = msg.data if msg.buffer_id == ofproto.OFP_NO_BUFFER else None
out = parser.OFPPACKETOut(datapath=datapath, buffer_id=msg.buffer_id,
                           in_port=in_port, actions=actions, data=data)
datapath.send_msg(out)

```

8. Arrancar o controller num terminal (usando o ambiente virtual criado):

```
$ ~/ryu-env/bin/ryu-manager learning_switch.py
```

9. Noutro terminal, arrancar o Mininet a apontar para este controller externo (por omissão, escuta em 127.0.0.1:6633):

```
$ sudo mn --topo single,3 --mac --switch ovsk \
    --controller remote,ip=127.0.0.1,port=6633
mininet> pingall
```

Confirmar que a conectividade funciona novamente, agora com as regras a serem instaladas pelo nosso controller.

10. Inspeccionar a tabela de fluxos instalada no switch (executar num terminal fora do Mininet, com a topologia ainda a correr):

```
$ sudo ovs-ofctl dump-flows s1
```

Identificar as entradas *match* – *action* correspondentes aos pares de hosts que já comunicaram. Para cada entrada, localize também os elementos discutidos na teoria:

```

- priority=... (qual regra vence quando varias correspondem)
- n_packets=... (contador de pacotes que corresponderam)
- n_bytes=... (contador de bytes)
- duration=... (ha quanto tempo a entrada existe)

```

Gere mais tráfego e volte a consultar a tabela, confirmando que os contadores aumentam:

```

mininet> h1 ping -c 5 h2
$ sudo ovs-ofctl dump-flows s1

```

11. Modificar o controller para bloquear especificamente o tráfego entre **h1** e **h3**, mantendo a comunicação com **h2**: no método `packet_in_handler`, antes de calcular `out_port`, adicionar uma verificação sobre os endereços MAC de origem/destino conhecidos de **h1** e **h3** (obter os MACs com `h1 ifconfig` e `h3 ifconfig` dentro da consola do Mininet) e, nesse caso, instalar uma regra com lista de `actions` vazia (`drop`) em vez de reencaminhar.
12. Reiniciar o controller e o Mininet com a alteração, e confirmar o novo comportamento:

```
mininet> h1 ping -c 3 h2
mininet> h1 ping -c 3 h3
```

## 6 Entregável / Questões de Verificação

---

- a) Enviar o ficheiro `learning_switch.py` final (com o bloqueio **h1-h3**) e a saída de `ovs-ofctl dump-flows s1` antes e depois da alteração.
- b) Que bridge do Proxmox liga as VMs da turma, e a que interface física corresponde? Em que sentido é ela um "switch virtual"?
- c) Registe a sub-rede dos Pods e a sub-rede das VMs observadas na Parte 1. Qual é o *overlay* e qual é o *underlay*? Que interface faz o papel de VTEP?
- d) Porque é que o `pingall` falha quando não existe nenhum controller ligado ao switch? O que isto demonstra sobre a relação entre plano de controlo e plano de dados?
- e) Que informação usa o controller para decidir a porta de saída de um pacote (`mac_to_port`)? Que camada do modelo OSI está a ser usada nessa decisão?
- f) O controller implementado neste guião segue um modelo reativo ou proativo? Justifique com base no que acontece quando chega um pacote sem regra correspondente.
- g) Observou os contadores (`n_packets`) a aumentar nas entradas de fluxo. Para que serve, na prática, esta informação num datacenter real?
- h) Em que consiste, neste guião, o plano de dados e em que consiste o plano de controlo? Identifique claramente os dois componentes usados.